

DEBUGMATE

AI-Powered Beginner-Friendly Debugging Assistant

Team Name: AI Avengers

Team Members:

Srudhya S – E25AI235

Kaviya Shree V – E25AI220

Hemalatha K – E25AI213

Lenita Freesia T – E24CO203

Table of Contents:

S.no	Title
1	Abstract
2	Introduction
3	Project Overview
4	Objectives
5	Problem Statement
6	Tools & Technologies Used
7	System Workflow / Architecture
8	Implementation
9	Screenshots with Explanations
10	Testing & Test Cases
11	Output Screenshots
12	Challenges, Solutions & Limitations
13	Future Enhancements
14	Conclusion
15	References

Abstract:

Debug Mate is an AI-powered debugging assistant designed to help beginner programmers understand and fix programming errors instead of simply providing solutions.

Beginners frequently struggle with syntax errors, logical bugs, and runtime exceptions. They may find error messages hard to understand and often rely on search engines or others to solve their problems. This can create learning gaps and reduce their ability to debug independently.

Debug Mate addresses this problem by analysing programming errors and explaining them in simple, beginner-friendly language. The system identifies the location of a bug, provides hints before revealing the complete solution, and explains the programming concept behind the error.

Debug Mate's main goal is to turn debugging from a problem-solving task into a learning opportunity.

Introduction:

Debugging is an essential part of programming. A programmer must be able to identify errors, understand their causes, and modify the program to obtain the expected result.

However, beginners often find debugging difficult because programming error messages can be confusing. Instead of understanding the cause of an error, students may simply copy the error message into a search engine and use an available solution.

Debug Mate is designed to encourage a better learning approach. It uses AI to understand the context of the code and guide students through the debugging process.

The system focuses on helping students learn **why an error occurs**, **where it occurs**, and **how it can be fixed** rather than immediately giving them the answer.

Project Overview:

Debug Mate is an AI-based debugging assistant for beginner programmers. The system provides four important capabilities:

1. **Error Explanation** – Converts complex error messages into simple language.
2. **Bug Localisation** – Identifies the specific line, statement, or logic responsible for the error.
3. **Hint-First Mode** – Gives students a hint before revealing the complete solution.
4. **Full-Fix Mode** – Provides corrected code when the student requests the solution.

These features are intended to make debugging more educational and interactive.

Objectives:

The main objectives of Debug Mate are:

5.1 Explain Programming Errors

The system should detect and explain programming errors using simple, beginner-friendly language.

5.2 Localise the Bug

Debug Mate should identify the exact line, statement, or logic responsible for the problem.

5.3 Provide Hints Before Solutions

Instead of immediately displaying the corrected program, the system should provide hints that encourage students to think about the problem independently.

5.4 Build Root-Cause Understanding

The system should help students understand the programming concept behind the error so that they can avoid similar mistakes in the future.

Problem Statement:

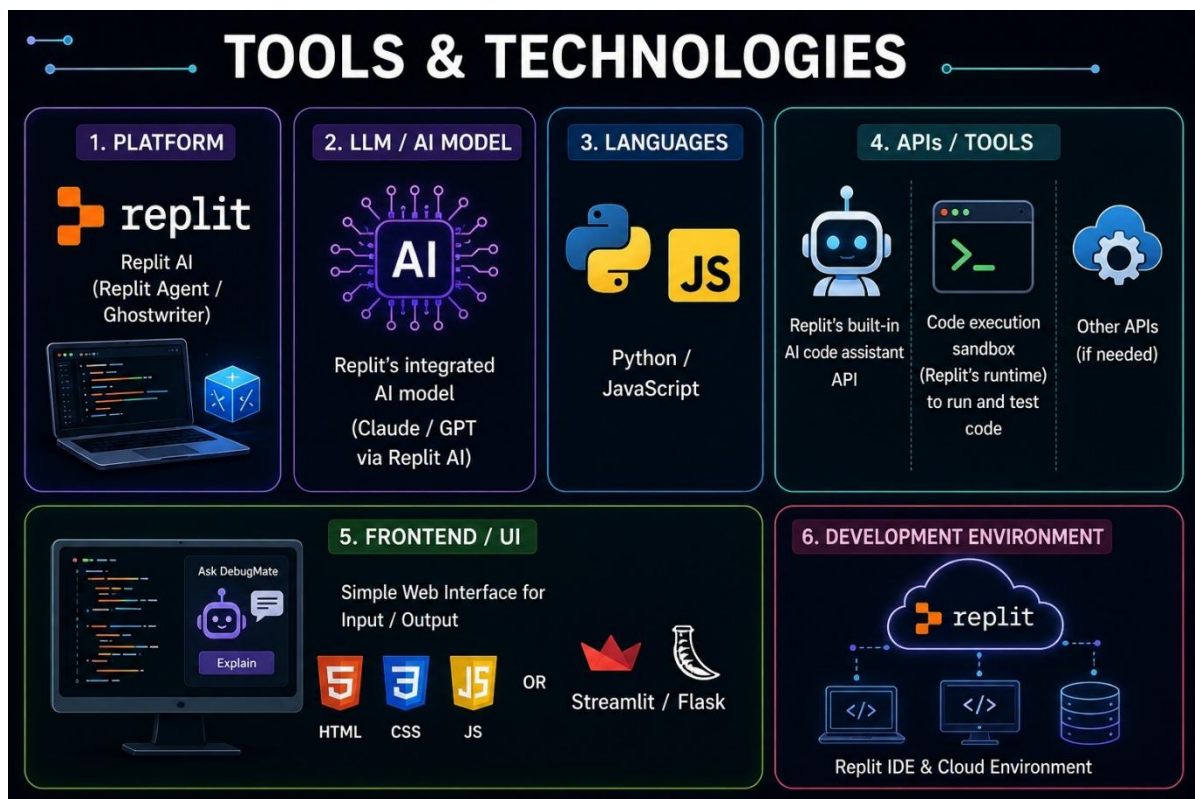
Beginners often struggle with programming errors such as syntax errors, logical bugs, and runtime exceptions. They may also find programming error messages difficult to understand.

Another major problem is that students may copy error messages into search engines or ask others for solutions without understanding why the error occurred.

This creates learning gaps and dependency on external help. As a result, students may learn how to fix one particular error without developing proper debugging skills.

Debugging is an important part of developing real programming understanding. Therefore, there is a need for a beginner-friendly system that teaches students how to understand and debug their errors rather than simply fixing them.

Tools & Technologies Used:



System Workflow / Architecture:

The basic workflow of Debug Mate can be represented as:



Main Components:

1. User Input

The student provides the programming code and/or error information.

2. AI Analysis

The system analyses the code and error context.

3. Error Explanation

The complex error is converted into beginner-friendly language.

4. Bug Localisation

The system identifies the relevant line, statement, or logic.

5. Hint Generation

The student receives a hint before the complete solution.

6. Full Fix

If required, the system provides corrected code and explains the changes.

Implementation:

Debug Mate is implemented as an AI-assisted debugging system.

The implementation focuses on making debugging understandable for beginners.

Step 1: Receive the Code

The user provides the program or problematic code.

Step 2: Analyse the Error

The AI examines the code and the available error information to understand the programming problem.

Step 3: Explain the Error

Instead of presenting only the technical error message, Debug Mate explains the problem using simple language.

Step 4: Locate the Bug

The system identifies the particular line, statement, or logical section responsible for the problem.

Step 5: Provide a Hint

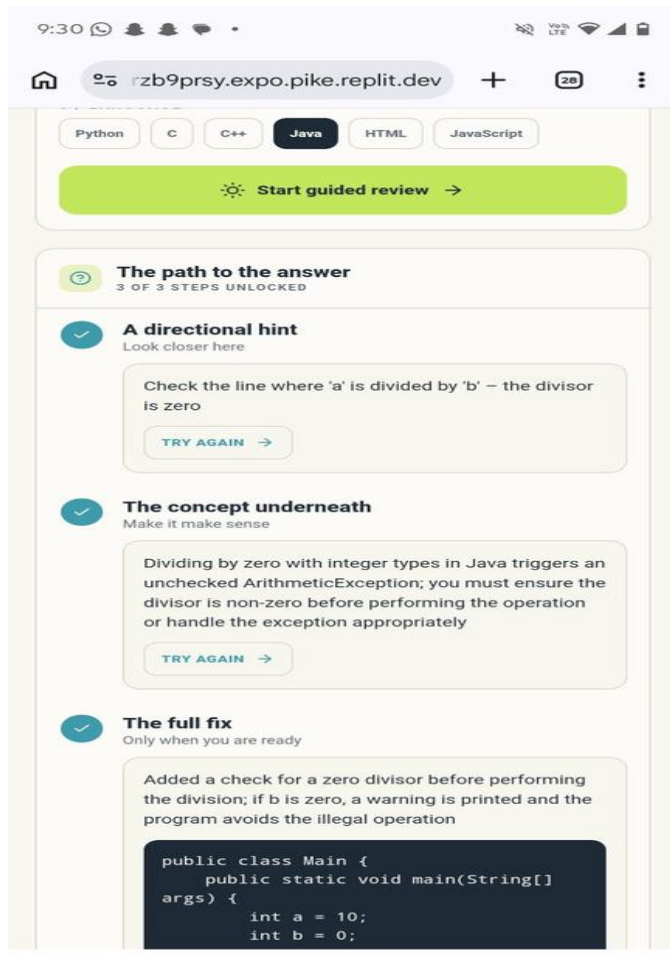
The system gives the student a hint so that they can attempt to identify the solution themselves.

Step 6: Provide the Complete Fix

When the student needs additional help, Debug Mate provides the corrected code and explains what was changed.

Screenshots with Explanations:





Explanation:

1. Directional Hint

Look at the division line.

The program is dividing a by b. Check what value is stored in b.

2. The Concept Underneath

Why is this a problem?

b has a value of 0, so the program is trying to calculate:

$$10 \div 0$$

Java does not allow dividing an integer by zero. This causes an Arithmetic Exception and stops the program.

3. The Full Fix

Check the divisor before dividing.

If b is zero, show an error message. Otherwise, safely perform the division.

```
if (b == 0) {
    System.out.println("Error: Division by zero is not
allowed.");
} else {
    int result = a / b;
    System.out.println("Result: " + result);
}
```

What changed?

We added a check before a / b, so the program no longer tries to divide by zero.

In simple words, we can,

Where is the bug? → Why is it a bug? → How do I fix it?

Test Case:

**TEST CASE SUMMARY**
Division by Zero (All Languages)

PASS

TEST CASE ID
TC-DZ-001

BUG TYPE
Division by Zero

LANGUAGES
Python, C, C++,
Java, HTML,
JavaScript

OBJECTIVE
Detect and handle
division by zero and
provide a fix.

DATE
25 May 2025

Language	Buggy Code	Issue / Error	Expected Result	Fix / Corrected Code	Status
 Python	<pre>x = 10 / 0 print(x)</pre>	 ZeroDivisionError: division by zero	Handle zero divisor and avoid runtime error.	<pre>x, y = 10, 0 y = y if y != 0 else None if y is None: print('Error: Division by zero') else: print(x / y)</pre>	 PASS
 C	<pre>#include <stdio.h> int main(){ printf("%d", 10 / 0); return 0; }</pre>	 Runtime Error (Illegal division by zero)	Ensure divisor is non-zero before performing division.	<pre>#include <stdio.h> int main(){ int b = 2; // non-zero printf("%d", 10 / b); return 0; }</pre>	 PASS
 C++	<pre>#include <iostream> using namespace std; int main() { cout << 10 / 0; return 0; }</pre>	 Runtime Error (Illegal division by zero / Crash)	Use a non-zero denominator or validate before division.	<pre>#include <iostream> using namespace std; int main() { int d = 2; // non-zero cout << 10 / d; return 0; }</pre>	 PASS
 Java	<pre>public class Main { public static void main (String[] args){ System.out.println(10/0); } }</pre>	 ArithmeticException: / by zero	Check denominator before performing division.	<pre>public class Main { public static void main (String[] args) { int d = 2; // non-zero System.out.println(10 / d); } }</pre>	 PASS
 HTML	<pre><html> <h1>Welcome</h1> <p>This is a paragraph< </html></pre>	 Unclosed <p> tag (Malformed HTML)	Close all opened tags properly.	<pre><!DOCTYPE html> <html> <head><title>Welcome Page</title></head> <body> <h1>Welcome</h1> <p>This is a paragraph</p> </body> </html></pre>	 PASS

VALIDATION CHECKLIST

Bug identified

Hint generated

Concept explained

Fix provided

Fix verified

ALL TEST CASES PASSED

Output Screenshots:

 Debugmate

Run Once

Run Batch

Code

```
<!DOCTYPE html>
<html>
<head>
<title>H1 Error Example</title>
</head>
<body>
```

Error

Mismatched Closing Tag Error

Stage

Hint

Language

HTML

Clear

Execute

POWERED BY

Dify

Workflow Process

USER INPUT

DEBUGMATE

OUTPUT

RESULT

DETAIL

> Thought(0.0s)

Hint

Check the tags you open and close.

In your code you start an `<h1>` element, but you close it with `</h2>`.

Closing tags must exactly match the opening tag's name.

Make sure the closing tag is `</h1>` (or change the opening tag to `<h2>` if that was your intention).

Workflow Process

USER INPUT

DEBUGMATE

OUTPUT

RESULT

DETAIL

> Thought(0s)

Concept: Matching Opening and Closing Tags in HTML

In HTML, most elements are written with a pair of tags:

- Opening tag – starts the element, e.g., `<h1>`.
- Closing tag – ends the element, e.g., `</h1>`.

The two tags must have the same name (except for the forward slash in the closing tag). Think of it like a pair of parentheses: every opening parenthesis needs a matching closing parenthesis.

In your code you opened an `<h1>` element:

```
HTML
<h1>Welcome to My website
```

but you closed it with `</h2>` instead of `</h1>`:

```
HTML
<h1>
</h2>
```

Because the tag names don't match, the browser reports a Mismatched Closing Tag error.

Why it matters

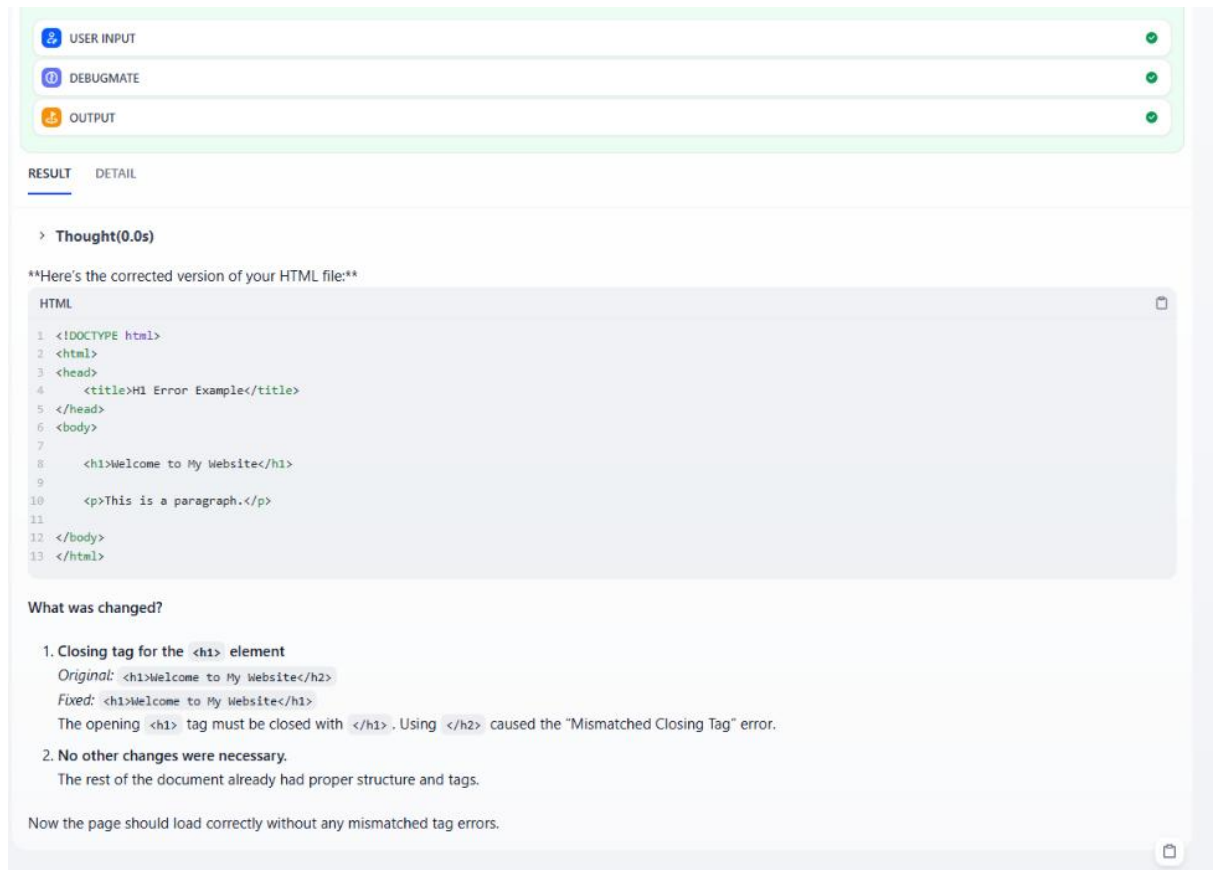
- Readability – Other people (and your future self) can quickly see where an element starts and ends.
- Correct rendering – Browsers use the tag names to determine how to display content. A wrong closing tag can cause the heading to be rendered as normal text, or the browser to "guess" and fix the mistake in unpredictable ways.
- Validation – Tools that check your HTML (like W3C Validator) will flag mismatches, which can help you catch other problems early.

Quick fix

Make the opening and closing tags match:

```
HTML
<h1>
<h1>Welcome to My website</h1>
```

Now the heading starts and ends with the same tag, and the browser will render it as a level-1 heading.



Challenges & Solutions, Limitations:

Challenges & Solutions

- Challenge: Explaining errors without overwhelming beginners.
Solution: Responses are staged (Hint → Concept → Fix) so students see only as much detail as they are ready for.
- Challenge: Preventing students from skipping straight to the answer.
Solution: The full fix is withheld until the hint and concept stages have been engaged with, encouraging independent thinking.
- Challenge: Supporting multiple programming languages consistently.
Solution: A shared Debugging Logic Layer normalises AI output into the same three-stage structure regardless of language.

Limitations:

- Large Codebases — Debug Mate may struggle to analyse very large or multi-file projects.
- AI Accuracy — responses depend on the reasoning and accuracy of the underlying AI model.
- Not a Teacher Replacement — students still need hands-on programming and debugging practice.
- Internet Dependency — requires internet / API access for AI-powered functionality.

Advantages:

- Active Learning — encourages students to understand and solve bugs instead of copy-pasting solutions.
- Instant Feedback — provides debugging guidance anytime without waiting for a mentor.
- Beginner-friendly — explains complex error messages in simple, easy-to-understand language.
- Personalised Guidance — analyses the student's specific code and provides customised explanations.
- Multiple Error Types — can assist with syntax, runtime, and logical errors.

Future Enhancement:

- Visual Debugger — add a step-by-step debugger that visually shows variable values and program flow.
- Personalised Learning Analytics — track students' progress and common error patterns to provide personalised guidance.
- Voice-Based Assistance — add voice explanations to make debugging more accessible and interactive.

- IDE Integration — extend Debug Mate beyond Replit with a VS Code extension and other development environments.

Conclusion:

Debug Mate transforms debugging into a learning opportunity by using Replit AI to understand code contextually, explain concepts, and guide students toward the solution. It addresses the gap in programming education by teaching students how to debug rather than simply fixing their bugs for them.